

Trabajo Practico 1

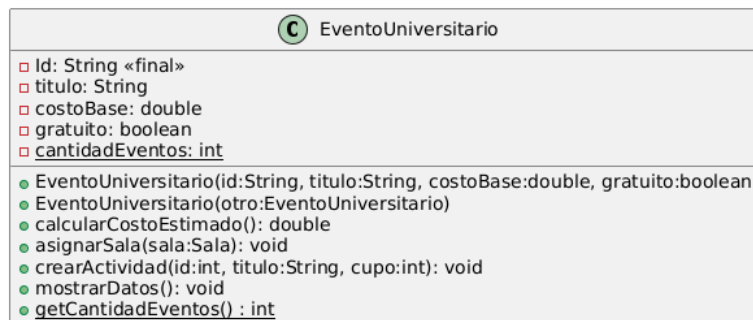
Programación Orientada a Objetos en Java

Unidad 1 - Fundamentos de la POO e implementación básica en Java

Ejercicio 1

Contexto: La universidad necesita contar con un sistema para administrar eventos simples como charlas, jornadas, talleres, hackathones, competencias, etc. Por ahora solo se necesita registrar los datos individuales de cada evento, sin mantener listas de participantes ni vincularlos a otro tipo de entidades.

Se requiere: implementar correctamente en Java la siguiente clase que modela un evento universitario, manejando adecuadamente los niveles de encapsulamiento requeridos (con calificadores de acceso) y propiedades de cada atributo y método. La aplicación debe permitir crear eventos correctamente inicializados, copiar eventos y consultar información de clase.



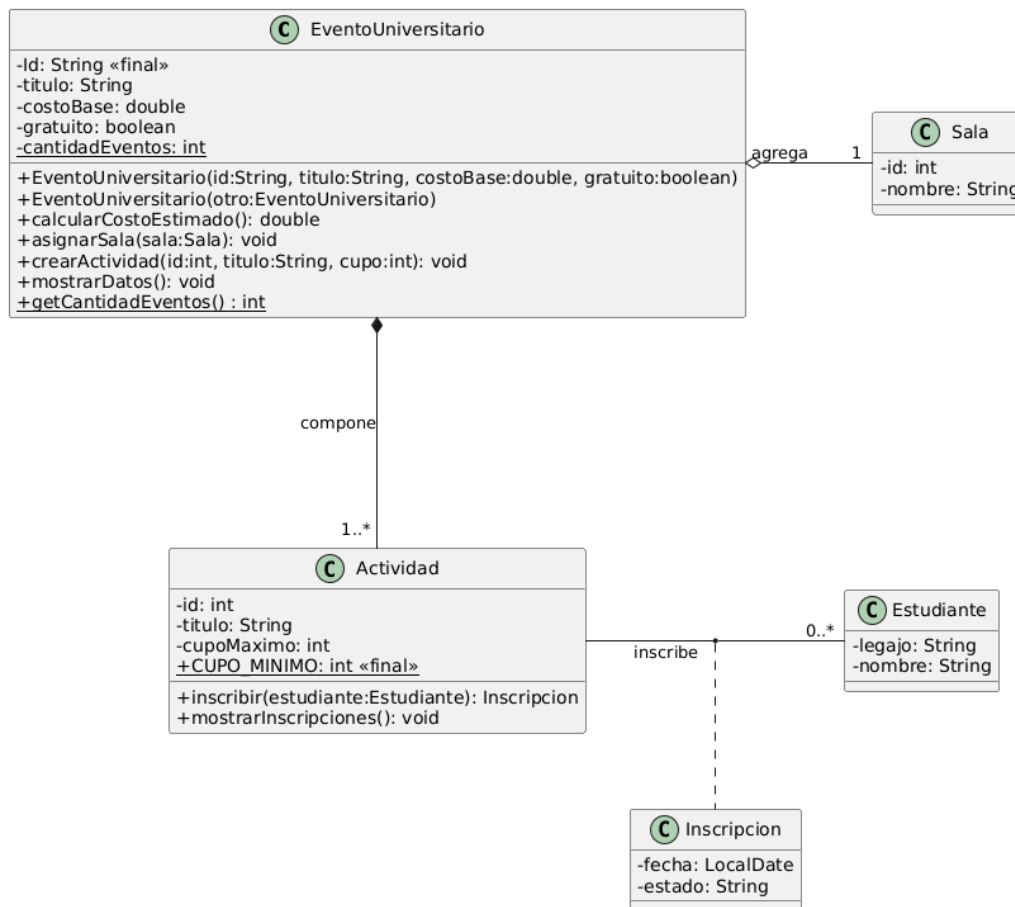
Resultado esperado: Código Java funcional ejecutable desde una clase App donde:

- Se creen uno o más eventos universitarios.
- Se cree una copia de cada evento creado utilizando el constructor de copia.
- Se muestren los datos de los eventos creados y su copia.
- Se muestre el contador de eventos con la totalidad de eventos creados.

Ejercicio 2

Contexto: Cada evento ya no puede pensarse como una clase aislada: se relaciona con una agenda de actividades, una sala asignada y estudiantes inscriptos en cada una de las actividades.

Se requiere: implementar correctamente en Java, el siguiente diagrama de clases, que escala el modelo del ejercicio anterior incorporando relaciones entre clases y objetos, de manera que el sistema pueda crear eventos, asignar a cada evento una sala, registrar sus actividades y la lista de alumnos inscriptos en cada actividad.



Para poder consultar las inscripciones de una actividad, agregar en Actividad una colección List<Inscripcion> inscripciones.

Resultado esperado: Código Java funcional ejecutable desde una clase App donde:

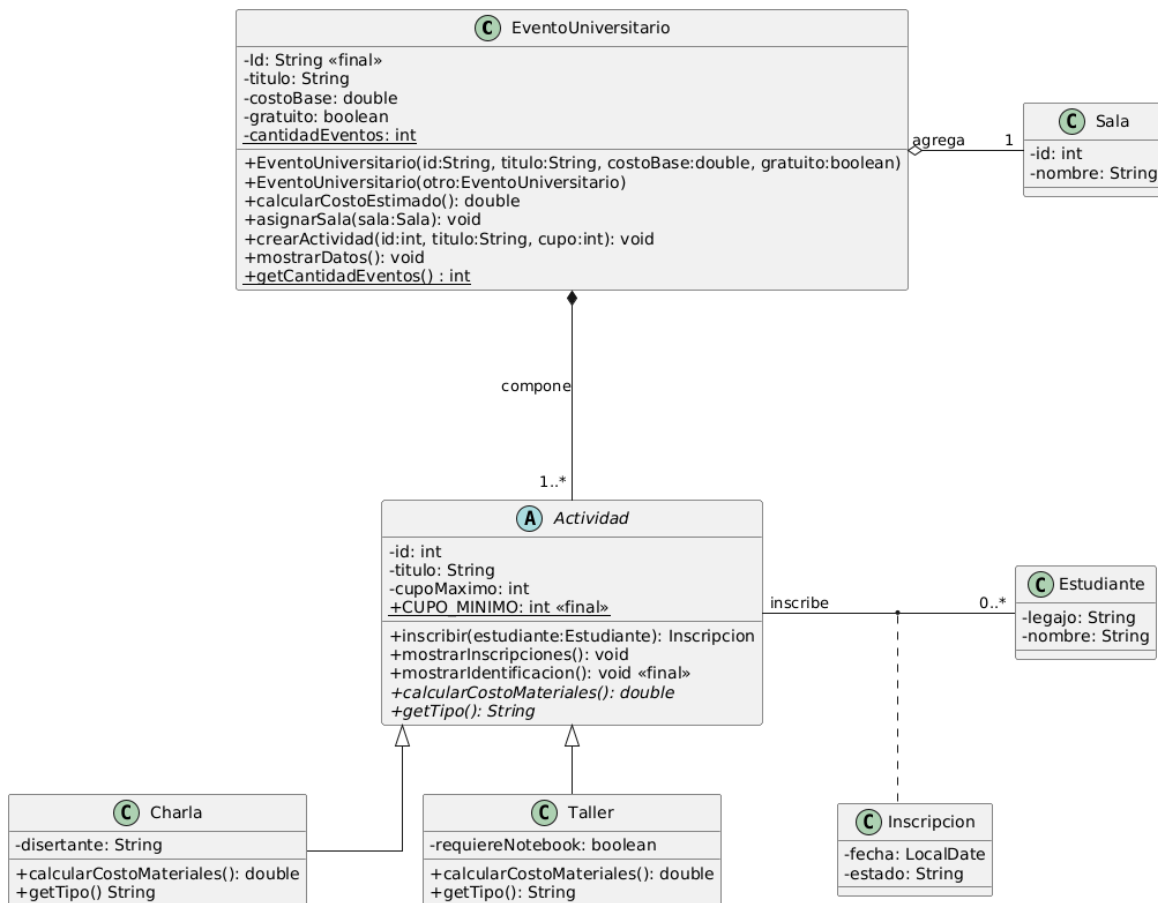
- Se construya una lista de estudiantes.
- Se construyan eventos.
- Se asigne una sala cada evento.
- Se creen actividades propias de cada evento.
- Se inscriban estudiantes en cada actividad.
- Se muestre el resumen de datos por cada evento creado.
- Se muestre el total de eventos creados.

Ejercicio 3

Contexto: Cada evento universitario incorpora actividades de distinta naturaleza. Una charla, un taller o a futuro, un curso, una competencia, una hackathon, etc. Estos tipos de actividades comparten datos comunes, pero difieren en su forma de calcular su costo y mostrar su identificación.

Se requiere: implementar correctamente en Java, el siguiente diagrama de clases, que escala el modelo del ejercicio anterior incorporando relaciones de herencia y polimorfismo, de manera que

el sistema pueda tratar diferentes tipos de actividades de manera polimórfica garantizando mejor reutilización de código y escalabilidad futura. El modelo también ilustra el uso de métodos finales.



Notar que la clase *Actividad*, que en el Ejercicio 2 era una clase concreta, debe transformarse en una clase abstracta porque ya no se instanciarán actividades genéricas, sino tipos concretos de actividad (Charla o Taller).

Notar que el método `mostrarIdentificacion()` de la clase *Actividad* se califica como final para que no pueda redefinirse en las subclases.

En la clase *EventoUniversitario* mantener la colección `List<Actividad>` actividades, pero ahora esa colección almacenará objetos de subclases concretas: *Charla* y *Taller*. En relación a esto deberá modificarse el método `crearActividad` para que reciba también, mediante un parámetro del tipo `String`, el tipo de actividad a crear: "Charla" o "Taller".

En la clase *EventoUniversitario*, deberá modificarse también el método `calcularCostoEstimado`. Si el evento es gratuito, el costo total deberá seguir siendo cero. En otro caso, el costo del evento deberá ser el (costoBase + el costo de cada una de sus actividades) * 1,21 para seguir incorporando el 21% en concepto de impuestos.

Respecto a la forma de calcular el costo de cada tipo de actividad, considerar que las charlas son gratuitas y los talleres tienen un costo de \$5000 si requieren uso de notebook y \$2000 si no requieren uso de notebook.

Resultado esperado: Código Java funcional ejecutable desde una clase App donde:

- a. Se registren estudiantes.
- b. Se construyan eventos.
- c. Se asigne una sala a cada evento.
- d. Se creen actividades para cada evento del tipo Charla y/o Taller.
- e. Se inscriban estudiantes en cada actividad.
- f. Se muestre el resumen de datos de cada evento y se recorran sus actividades mostrando su identificación de forma polimórfica.
- g. Se muestre el total de eventos creados.

Ejercicio 4

Contexto: Es relevante que, al ejecutar un programa orientado a objetos, el estudiante comprenda cómo se construyen y vinculan los objetos en memoria durante la ejecución. Esto implica reconocer cómo se materializan los distintos tipos de relaciones entre objetos, diferenciar referencias, asociaciones, agregaciones, composiciones y relaciones de herencia, y comprender de qué manera se crean, mantienen y dejan de estar accesibles los objetos en memoria.

Se requiere:

A partir del programa implementado en el Ejercicio 3 y suponiendo que:

- a. Se crean 3 estudiantes.
- b. Se crea 1 evento.
- c. Se crea 1 sala.
- d. Se crean 2 actividades para el evento: una Charla y un Taller.
- e. Se inscriben 2 estudiantes en la Charla.
- f. Se inscriben 2 estudiante en el Taller.

Elaborar un mapa de memoria de ejecución –en formato gráfico- que represente gráficamente qué objetos se crean durante la ejecución del método main de la clase App y cómo quedan vinculados entre sí mediante referencias.

El mapa deberá mostrar, como mínimo:

1. Las variables locales declaradas en el método main, indicando a qué objetos hacen referencia. Solo considerar variables que referencian a objetos del tipo Estudiante, EventoUniversitario, Sala y sus relaciones.
2. El objeto EventoUniversitario creado en la ejecución.
3. El objeto Sala asignado al evento, identificando que se trata de una relación de agregación, ya que la sala existe independientemente del evento.
4. La colección de actividades del evento, mostrando que el evento contiene objetos de tipo Charla y Taller, representados como subclases de Actividad.
5. La relación de composición entre EventoUniversitario y sus actividades, indicando que las actividades forman parte del evento.
6. Los objetos Estudiante creados en la ejecución.
7. Los objetos Inscripcion generados al inscribir estudiantes en cada actividad.

8. Las referencias que vinculan cada Inscripción con la actividad correspondiente y con el estudiante inscripto.
9. La estructura de herencia entre Actividad, Charla y Taller, indicando que los objetos Charla y Taller contienen la parte heredada de Actividad y su parte específica de subclase.

El mapa puede realizarse con cualquier herramienta de dibujo digital. Debe diferenciar claramente el espacio de variables locales del método main y el espacio en el heap donde se encuentran los objetos construidos en memoria.

Resultado esperado: una imagen del mapa de memoria que permita explicar cómo se construyen y vinculan los objetos del modelo durante la ejecución del programa, evidenciando las diferencias entre asociación, agregación, composición y herencia.

PAUTAS DE ENTREGA DEL TP

Para que la tarea se considere entregada debe:

1. Crear un repositorio en GitHub, de acceso público, cuyo nombre sea: PP_TP1_legajo

Por ejemplo: https://github.com/jperez/PP_TP1_50268

2. El repositorio debe contener lo siguiente:
 - 2.1. El proyecto de código desarrollado, completo hasta el ejercicio 4, generado desde INTELLIJ IDEA listo para clonar y probar.
 - 2.2. Un Readme con documentación sobre el proyecto implementado.
 - 2.3. La imagen del gráfico solicitado en el punto 4.
 - 2.4. Una captura de la salida por consola de una ejecución del programa.
3. Consignar en la entrega, la URL para clonar el repositorio vía https. (En el enlace Entregar Trabajo Práctico N° 1).

ATENCIÓN tener en cuenta:

- lo indicado anteriormente es requisito para que la tarea sea evaluada (VERIFICAR que se cumple con todos los requisitos ANTES de concretar la entrega, ya que en caso de que no se pueda clonar el repositorio, falte alguno de los archivos o no coincida en formatos o nombres con lo solicitado, no se califica y NO se otorga tiempo adicional para completar la entrega.
- COMPROBAR que el repositorio se puede clonar correctamente sin requerir permisos.
- SI SE COMPRUEBAN MODIFICACIONES EN EL REPO POSTERIORES A LA FECHA DE CIERRE DE LA ENTREGA EL TP SERÁ CONSIDERADO NO ENTREGADO.